

Production-grade Supabase Backend Execution Plan

Executive summary

The strongest production pattern with Supabase is a layered backend in which PostgreSQL is the system of record, Row Level Security is the primary authorization layer, PostgREST handles commodity CRUD and list endpoints, Edge Functions handle privileged workflows and third-party integrations, Realtime handles event delivery, and Storage handles file assets under the same identity model. Supabase's own architecture and product docs support this shape: Auth issues JWTs that scope access to autogenerated database APIs, GraphQL and Realtime both sit on top of the same Postgres-backed security model, and Edge Functions provide a serverless execution layer for orchestration and integrations. ¹

For an AI to implement this backend autonomously, the best default is a **REST + RPC first** design, with GraphQL added only where it provides clear value for nested dashboards or partner-facing query ergonomics. PostgREST already gives filtering, ordering, embedding, HTTP-range pagination, and `/rpc` function endpoints; Supabase's GraphQL endpoint is useful, but it is a second query surface that needs governance and should not be the first abstraction the AI reaches for. Likewise, Realtime should default to **Broadcast**, because Supabase explicitly recommends Broadcast over Postgres Changes for scalability and security. ²

The largest implementation risk is not CRUD complexity; it is **authorization drift**. The AI should therefore keep authorization facts in relational tables such as memberships, roles, and permissions, and use JWT custom claims only for coarse hints and optimized checks. Supabase documents that data read from `auth.jwt()` can become stale until the token refreshes, and that only non-user-editable app metadata should influence authorization. The design below reflects that: all sensitive permissions are evaluated against tables, not against mutable client assertions. ³

Operationally, the minimum production bar should include migrations-as-code, deterministic seeds, local stack parity, preview branches, database and function tests, PITR or daily backups, metrics, logs, external alerting, and careful secret handling. Supabase provides the primitives for that: CLI-managed local environments, `db lint`, `db reset`, `test db`, preview branches, daily backups, PITR, Logs Explorer, log drains, and a Prometheus-compatible metrics API. ⁴

The uploaded materials describe a portal-like product with listings, categories, advertisements and packages, subscriptions, consultations, admin roles, dashboards, and reporting. The concrete schema, API surface, and workflows below therefore assume that business model rather than a neutral example app.

filecite turn0file0 filecite turn0file1

Assumptions and guiding decisions

The uploaded notes point toward a multi-tenant portal or directory product with listing management, category hierarchies, consultation workflows, admin dashboards, advertising placements, subscription packages, and reporting. That is the domain model used below. If your actual product differs, the platform structure still holds, but the bounded-context tables should be swapped while keeping the same Supabase control plane, migration strategy, and RLS-first model. [filecite:turn0file0](#) [filecite:turn0file1](#)

Explicit assumptions

Assumption	Chosen value	Why it matters
Product shape	Multi-tenant portal SaaS	Makes tenant isolation first-class
Scale target	100k users, 10k peak concurrent connections	Drives Realtime, compute, and cost choices
Primary region	Nearest supported region to dominant users	Reduces latency and egress
Auth modes	Password, magic link/OTP, social OAuth, optional enterprise SSO	Covers both B2C and B2B
API style	REST + RPC first, GraphQL optional	Best fit for autonomous implementation
Storage default	Private buckets + signed URLs	Strong default confidentiality
Payments	Stripe	Best-supported recurring billing/webhook path
Email	Custom SMTP for Auth + provider-backed app email via Functions	Production deliverability
Compliance baseline	GDPR/CCPA-ready, HIPAA optional	Avoids premature overbuild
Search	PostgreSQL full-text search first	Keeps version one simpler and cheaper

Supabase deploys each project into one primary region and recommends choosing the location closest to users. For the stated concurrency target, plan limits matter: the Realtime limits page shows that ordinary Pro is capped at 500 concurrent connections, while “Pro (no spend cap)” and higher tiers raise that limit substantially; supporting 10k peak concurrent sockets therefore requires Pro with spend cap disabled or a higher plan. ⁵

Guiding principles for an autonomous AI implementer

The AI should treat every user-facing table in an exposed schema as a security boundary. Supabase explicitly recommends RLS for tables in exposed schemas, and once RLS is enabled, access is effectively default-deny until policies are added. The AI should also keep security-definer helper functions in a non-

exposed schema such as `private`, because Supabase advises against placing security-definer functions in exposed schemas. ⁶

The AI should follow a strict **code-first, dashboard-last** workflow. Local environments should be started from the CLI; migrations should be committed and replayable; seeds should be deterministic; functions should be developed locally and deployed from CI; preview branches should be created for PRs; and rollback should be driven by forward-fix migrations or restore procedures, not by ad hoc manual edits. That discipline matters more than any one schema choice. ⁷

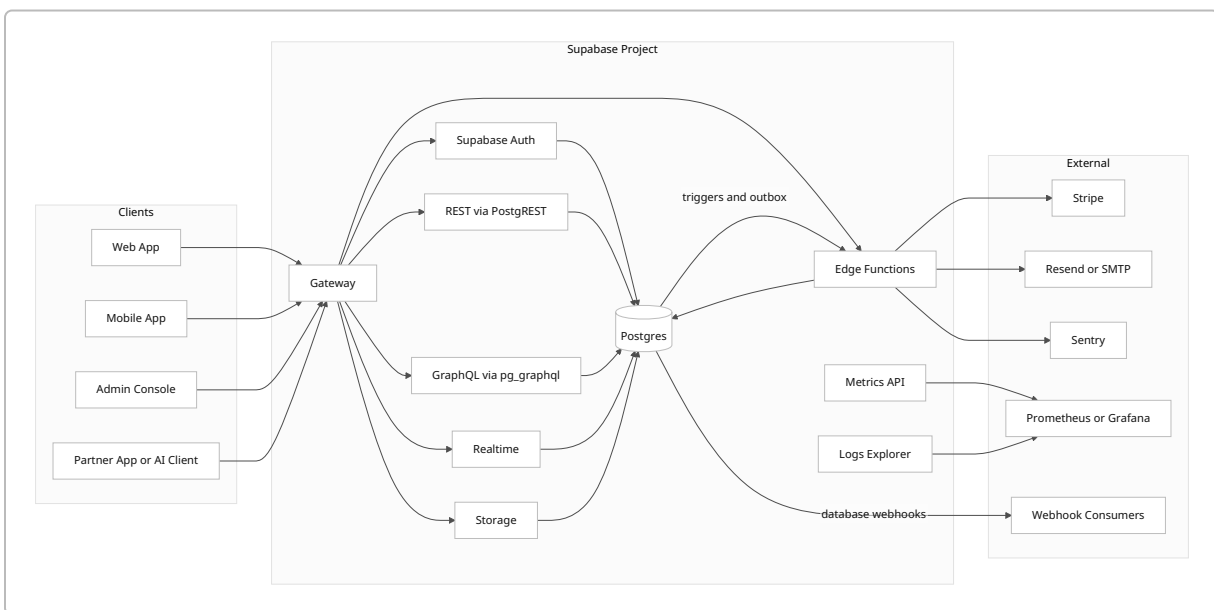
Open questions and limitations

One current limitation in this report is pricing presentation. Supabase's billing docs clearly document metered usage components and per-project compute pricing, but they point back to the live pricing page for the current fixed plan subscription fee. The cost model below therefore itemizes variable and per-project charges explicitly and treats the organization plan fee as `F_plan`, which should be pulled from the live pricing page at implementation time. ⁸

Reference architecture

Supabase's model is most effective when you let the managed components do the work they were designed to do. Auth owns identity and token issuance; PostgREST owns row-oriented CRUD and list APIs; `pg_graphql` is optional for nested reads; Realtime owns event fan-out; Storage owns files and object metadata; Edge Functions own privileged workflows, webhooks, and orchestrations; and PostgreSQL remains the single source of truth underneath all of them. ⁹

Architecture diagram



Trade-off tables

Supabase exposes autogenerated REST via PostgREST and autogenerated GraphQL via `pg_graphql`. PostgREST supports one-level resource routes, horizontal and vertical filtering, embedding based on foreign keys, range-based pagination, and `/rpc` functions; GraphQL is exposed at `/graphql/v1`, honors RLS, and is strong for deep reads. ¹⁰

API option	Strengths	Weaknesses	Recommendation
REST + RPC	Easier for AI to generate and test; native filters, ordering, pagination, embedding, and DB-side transactions	Nested dashboard reads can get verbose	Default
GraphQL	Excellent for nested dashboards and partner-facing developer ergonomics	More governance, second query surface, potential overfetch tuning needed	Add selectively
Edge Function-only API	Maximum contract control	Reimplements capabilities Supabase already gives you	Use only for privileged workflows

Supabase Realtime supports Broadcast, Presence, and Postgres Changes. The docs explicitly recommend Broadcast for scalability and security, describe Presence as a CRDT-backed in-memory state system, and note that Postgres Changes are simpler but less scalable. ¹¹

Realtime option	Best use	Notes	Recommendation
Broadcast	Notifications, consultation room events, status updates	Private channels are authorized via <code>realtime.messages</code> RLS	Primary
Presence	“Who is here” / “who is editing” state	Ephemeral and collaborative, not authoritative	Use with Broadcast
Postgres Changes	Small admin subscriptions, low-volume feeds	Simpler, but not the scale-first choice	Secondary only

Supabase Storage is designed for files, folders, and buckets, and Supabase explicitly recommends storing files outside the database due to size. Private buckets are private by default and can be accessed via JWT-authorized downloads or signed URLs. ¹²

Storage pattern	Best use	Access model	Recommendation
Private bucket + signed URL	User assets, invoices, reports, unapproved creatives	JWT/RLS + short-lived URLs	Default

Storage pattern	Best use	Access model	Recommendation
Public bucket	Approved public thumbnails, marketing files	Open read access	Use sparingly
Files in Postgres	Tiny edge cases only	Bloats backups and disk	Avoid for production media

Data model and normalized schema

The schema below is intentionally normalized around tenant identity, permissions, listings, categories, media, subscriptions, ad inventory, consultations, notifications, and event logging. PostgreSQL's documented integrity features—`NOT NULL`, `CHECK`, `UNIQUE`, `PRIMARY KEY`, and `FOREIGN KEY`—should carry as much correctness as possible, while partial indexes should enforce uniqueness or performance only where you truly query subsets of rows. GIN indexes are appropriate for full-text search, while B-tree remains the default for equality and range conditions. Supabase also recommends adding indexes to columns used by RLS predicates. ¹³

Supabase's `auth` schema is intentionally not exposed through the autogenerated API, so application-facing profile data should live in `public` and reference `auth.users`. Helper functions, idempotency state, outbox state, and audit tables should live in non-exposed schemas such as `private`, `internal`, and `audit`. This reduces accidental API exposure and keeps the RLS surface smaller. ¹⁴

Relationship summary

Parent	Child	Cardinality	Purpose
organizations	organization_memberships	One-to-many	Tenant membership and RBAC
role_catalog	role_permissions	One-to-many	System role → permission mapping
organizations	listings	One-to-many	Tenant-owned business records
listings	listing_categories	Many-to-many	Flexible taxonomy
listings	listing_media	One-to-many via bridge	Ordered attachments
organizations	organization_subscriptions	One-to-many	Billing history
ad_slots	ad_packages	One-to-many	Inventory slot pricing
ad_packages	ad_campaigns	One-to-many	Purchased campaigns
ad_campaigns	ad_creatives	One-to-many	Creative variants
organizations	consultations	One-to-many	Support/sales workflows
consultations	consultation_events	One-to-many	Immutable timeline

Parent	Child	Cardinality	Purpose
profiles	notifications	One-to-many	User inbox

Canonical core DDL

The AI should split this into multiple migration files, but this is the authoritative starting SQL.

```

create extension if not exists pgcrypto;

create schema if not exists private;
create schema if not exists internal;
create schema if not exists audit;

revoke all on schema private from anon, authenticated;
revoke all on schema internal from anon, authenticated;
revoke all on schema audit from anon, authenticated;

create type public.listing_status as enum (
    'draft', 'pending_review', 'published', 'archived', 'rejected'
);

create type public.consultation_status as enum (
    'open', 'scheduled', 'in_progress', 'resolved', 'cancelled'
);

create type public.campaign_status as enum (
    'draft', 'pending_review', 'approved', 'live', 'paused', 'completed',
    'rejected'
);

create type public.subscription_status as enum (
    'trialing', 'active', 'past_due', 'cancelled', 'expired'
);

create table public.profiles (
    id uuid primary key references auth.users(id) on delete cascade,
    display_name text not null,
    default_org_id uuid null,
    locale text not null default 'en-US',
    timezone text not null default 'UTC',
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now()
);

create table public.organizations (
    id uuid primary key default gen_random_uuid(),

```

```

slug text not null,
name text not null,
billing_email text null,
status text not null default 'active'
    check (status in ('active', 'suspended', 'closed')),
created_by uuid not null references auth.users(id),
created_at timestamptz not null default now(),
updated_at timestamptz not null default now()
);

create unique index organizations_slug_uidx
on public.organizations (lower(slug));

create table public.role_catalog (
    code text primary key,
    name text not null,
    description text null,
    is_system boolean not null default true
);

create table public.permission_catalog (
    code text primary key,
    description text not null
);

create table public.role_permissions (
    role_code text not null references public.role_catalog(code) on delete
cascade,
    permission_code text not null references public.permission_catalog(code) on
delete cascade,
    primary key (role_code, permission_code)
);

create table public.organization_memberships (
    id uuid primary key default gen_random_uuid(),
    org_id uuid not null references public.organizations(id) on delete cascade,
    user_id uuid not null references auth.users(id) on delete cascade,
    role_code text not null references public.role_catalog(code),
    is_active boolean not null default true,
    invited_by uuid null references auth.users(id),
    joined_at timestamptz not null default now(),
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now(),
    unique (org_id, user_id)
);

create index organization_memberships_user_idx
on public.organization_memberships (user_id, is_active);

```

```

create index organization_memberships_org_role_idx
  on public.organization_memberships (org_id, role_code, is_active);

create table public.categories (
  id uuid primary key default gen_random_uuid(),
  parent_id uuid null references public.categories(id) on delete restrict,
  slug text not null,
  name text not null,
  sort_order integer not null default 0,
  is_active boolean not null default true,
  created_at timestamptz not null default now()
);

create unique index categories_slug_idx
  on public.categories (lower(slug));

create index categories_parent_sort_idx
  on public.categories (parent_id, sort_order);

create table public.listings (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null references public.organizations(id) on delete cascade,
  owner_user_id uuid not null references auth.users(id),
  slug text not null,
  status public.listing_status not null default 'draft',
  title text not null,
  summary text null,
  description text null,
  website_url text null,
  contact_email text null,
  contact_phone text null,
  locality text null,
  administrative_area text null,
  country_code char(2) null,
  published_at timestamptz null,
  version integer not null default 1,
  search_vector tsvector generated always as (
    to_tsvector(
      'simple',
      coalesce(title, '') || ' ' ||
      coalesce(summary, '') || ' ' ||
      coalesce(description, '')
    )
  ) stored,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  unique (org_id, slug)
);

```

```

);

create index listings_org_status_idx
  on public.listings (org_id, status, created_at desc);

create index listings_published_idx
  on public.listings (published_at desc)
  where status = 'published';

create index listings_search_gin_idx
  on public.listings using gin (search_vector);

create table public.listing_categories (
  listing_id uuid not null references public.listings(id) on delete cascade,
  category_id uuid not null references public.categories(id) on delete restrict,
  primary key (listing_id, category_id)
);

create table public.media_assets (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null references public.organizations(id) on delete cascade,
  bucket text not null,
  object_path text not null,
  original_filename text not null,
  mime_type text not null,
  byte_size bigint not null check (byte_size >= 0),
  created_by uuid not null references auth.users(id),
  created_at timestamptz not null default now(),
  unique (bucket, object_path)
);

create index media_assets_org_created_idx
  on public.media_assets (org_id, created_at desc);

create table public.listing_media (
  listing_id uuid not null references public.listings(id) on delete cascade,
  media_asset_id uuid not null references public.media_assets(id) on delete
  cascade,
  sort_order integer not null default 0,
  is_primary boolean not null default false,
  primary key (listing_id, media_asset_id)
);

create unique index listing_media_primary_uidx
  on public.listing_media (listing_id)
  where is_primary = true;

create table public.plan_catalog (

```

```

id uuid primary key default gen_random_uuid(),
code text not null unique,
name text not null,
interval_code text not null check (interval_code in ('month', 'year')),
price_cents integer not null check (price_cents >= 0),
currency char(3) not null,
feature_flags jsonb not null default '{} '::jsonb,
is_active boolean not null default true,
created_at timestamptz not null default now()
);

create table public.organization_subscriptions (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null references public.organizations(id) on delete cascade,
  plan_id uuid not null references public.plan_catalog(id),
  provider text not null,
  provider_customer_id text null,
  provider_subscription_id text not null,
  status public.subscription_status not null,
  current_period_start timestamptz null,
  current_period_end timestamptz null,
  cancel_at_period_end boolean not null default false,
  metadata jsonb not null default '{} '::jsonb,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  unique (provider, provider_subscription_id)
);

create table public.ad_slots (
  id uuid primary key default gen_random_uuid(),
  code text not null unique,
  name text not null,
  page_key text not null,
  position_key text not null,
  is_active boolean not null default true
);

create table public.ad_packages (
  id uuid primary key default gen_random_uuid(),
  ad_slot_id uuid not null references public.ad_slots(id) on delete restrict,
  code text not null unique,
  name text not null,
  duration_days integer not null check (duration_days > 0),
  price_cents integer not null check (price_cents >= 0),
  currency char(3) not null,
  max_impressions integer null check (max_impressions is null or
max_impressions > 0),
  is_active boolean not null default true

```

```

);

create table public.ad_campaigns (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null references public.organizations(id) on delete cascade,
  listing_id uuid null references public.listings(id) on delete set null,
  ad_package_id uuid not null references public.ad_packages(id),
  status public.campaign_status not null default 'draft',
  name text not null,
  target_url text not null,
  starts_at timestamptz null,
  ends_at timestamptz null,
  approved_by uuid null references auth.users(id),
  approved_at timestamptz null,
  version integer not null default 1,
  created_by uuid not null references auth.users(id),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table public.ad_creatives (
  id uuid primary key default gen_random_uuid(),
  campaign_id uuid not null references public.ad_campaigns(id) on delete
cascade,
  media_asset_id uuid not null references public.media_assets(id) on delete
restrict,
  headline text null,
  body text null,
  cta_text text null,
  created_at timestamptz not null default now(),
  unique (campaign_id, media_asset_id)
);

create table public.consultations (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null references public.organizations(id) on delete cascade,
  listing_id uuid null references public.listings(id) on delete set null,
  requester_user_id uuid null references auth.users(id) on delete set null,
  assigned_to_user_id uuid null references auth.users(id) on delete set null,
  requester_name text null,
  requester_email text null,
  requester_phone text null,
  subject text not null,
  details text null,
  status public.consultation_status not null default 'open',
  scheduled_for timestamptz null,
  resolved_at timestamptz null,
  version integer not null default 1,

```

```

        created_at timestamptz not null default now(),
        updated_at timestamptz not null default now()
    );

create index consultations_org_status_idx
    on public.consultations (org_id, status, created_at desc);

create table public.consultation_events (
    id uuid primary key default gen_random_uuid(),
    consultation_id uuid not null references public.consultations(id) on delete
cascade,
    actor_user_id uuid null references auth.users(id) on delete set null,
    event_type text not null,
    payload jsonb not null default '{}'::jsonb,
    created_at timestamptz not null default now()
);

create table public.notifications (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references auth.users(id) on delete cascade,
    org_id uuid null references public.organizations(id) on delete cascade,
    notification_type text not null,
    payload jsonb not null default '{}'::jsonb,
    read_at timestamptz null,
    created_at timestamptz not null default now()
);

create index notifications_unread_idx
    on public.notifications (user_id, created_at desc)
    where read_at is null;

create table internal.idempotency_keys (
    id uuid primary key default gen_random_uuid(),
    scope text not null,
    idempotency_key text not null,
    request_hash text not null,
    response_code integer null,
    response_body jsonb null,
    created_at timestamptz not null default now(),
    expires_at timestamptz not null,
    unique (scope, idempotency_key)
);

create table internal.outbox_events (
    id uuid primary key default gen_random_uuid(),
    aggregate_type text not null,
    aggregate_id uuid not null,
    event_type text not null,

```



```

payload jsonb not null,
status text not null default 'pending'
    check (status in ('pending', 'processing', 'delivered', 'failed')),
attempts integer not null default 0,
last_error text null,
next_attempt_at timestamptz not null default now(),
created_at timestamptz not null default now(),
updated_at timestamptz not null default now()
);

create index outbox_pending_idx
on internal.outbox_events (status, next_attempt_at, created_at)
where status in ('pending', 'failed');

create table audit.activity_log (
    id uuid primary key default gen_random_uuid(),
    org_id uuid null references public.organizations(id) on delete set null,
    actor_user_id uuid null references auth.users(id) on delete set null,
    entity_type text not null,
    entity_id uuid null,
    action text not null,
    request_id text null,
    metadata jsonb not null default '{} '::jsonb,
    created_at timestamptz not null default now()
);

```

Why this schema works well on Supabase

This model keeps the API surface clean while preserving tenant isolation and future flexibility. Roles and permissions are tables rather than hard-coded claims, which supports richer enterprise RBAC. Listing categories, media attachments, and consultation events are normalized bridges or append-only tables, which makes RLS easier to reason about and reduces update anomalies. The only denormalized field is the generated full-text `search_vector`, which is justified by search performance and indexed with GIN. ¹⁵

For reporting, add views rather than duplicative reporting tables unless you truly need precomputed aggregates. On PostgreSQL 15+, create reporting views with `security_invoker = true` so they inherit underlying RLS instead of bypassing it. That is especially important for tenant-scoped dashboards. ¹⁶

Authentication, authorization, API, and realtime

Supabase Auth supports password, magic link, OTP, social login, phone login, passkeys, enterprise SSO, and optional OAuth 2.1/OIDC provider mode. It issues JWTs that can be sent automatically by Supabase client SDKs to the autogenerated REST API, and those JWTs are then enforced together with RLS inside PostgreSQL. Supabase also supports many social providers and custom OAuth2/OIDC-compatible providers. ¹⁷

Recommended role model

Layer	Mechanism	Purpose
Platform role	<code>anon</code> , <code>authenticated</code> , <code>service_role</code>	Transport and platform control
Identity	<code>auth.users</code> + <code>public.profiles</code>	User identity and app profile
Tenant membership	<code>organization_memberships</code>	Which organizations a user belongs to
RBAC	<code>role_catalog</code> , <code>permission_catalog</code> , <code>role_permissions</code>	Application authorization
Optional JWT custom claims	Access token hook / SSO attribute map	UI hints and coarse routing

Use `service_role` only in trusted server contexts. Supabase's JWT reference defines `anon`, `authenticated`, and `service_role` and describes `service_role` as server-side only with admin privileges. Prefer the newer JWT signing keys model over the legacy JWT secret, which Supabase does not recommend for new deployments. For custom claims, put authorization facts in `app_metadata`, not `user_metadata`, because `user_metadata` is user-editable. ¹⁸

Auth flows

For self-serve users, implement password, magic link/OTP, and social OAuth. For enterprise customers, add SAML or OIDC SSO and map SSO attributes into JWT claims where useful. Supabase CLI supports SSO provider management and lets you map SAML attributes into custom JWT claims. If you later need AI agents or partner apps to obtain user-scoped access tokens, enable the OAuth 2.1/OIDC provider mode and use the authorization code flow with PKCE. ¹⁹

A production auth flow should work like this:

1. User authenticates with Supabase Auth.
2. A JWT and refresh token are issued.
3. Browser/mobile apps call REST/GraphQL with the publishable key and bearer token.
4. Low-risk CRUD runs directly against PostgREST under RLS.
5. Privileged workflows go through Edge Functions, which validate input, inspect claims, and then use server credentials only where necessary.

For sensitive actions such as membership changes, billing changes, destructive admin operations, and personal-data export, require `aal2` and check it in RLS and/or the function layer. Supabase documents using JWT assurance levels this way. ²⁰

API specification

The AI should expose a **hybrid contract**:

- **Direct REST** for low-risk CRUD and list endpoints.
- **RPC** for multi-table transactional workflows that still belong in the database.
- **Edge Functions** for privileged orchestration and all third-party integrations.
- **Optional GraphQL** for dashboards or external developer platforms.

PostgREST exposes tables and views in one-level routes, supports filter operators and embedding, and uses range headers or `limit/offset` for pagination. Functions in exposed schemas are available under `/rpc`, and if they return table types they can participate in the same filtering and embedding model. 21

Recommended endpoints

Surface	Path	Purpose	Auth
REST	<code>/rest/v1/organizations</code>	Current user's tenant list	JWT
REST	<code>/rest/v1/listings</code>	Listing CRUD and list pages	Public for published read, JWT otherwise
REST	<code>/rest/v1/categories</code>	Public taxonomy	Public read
REST	<code>/rest/v1/consultations</code>	Tenant consultations	JWT
RPC	<code>/rest/v1/rpc/search_listings</code>	Shaped search result with ranking	Public/JWT
RPC	<code>/rest/v1/rpc/upsert_listing_with_categories</code>	Atomic listing write	JWT
GraphQL	<code>/graphql/v1</code>	Nested dashboard reads	JWT
Edge Function	<code>/functions/v1/onboard-org</code>	Org creation + membership seeding	JWT
Edge Function	<code>/functions/v1/create-checkout-session</code>	Stripe checkout/session	JWT
Edge Function	<code>/functions/v1/stripe-webhook</code>	Signed Stripe webhook receiver	Stripe secret
Edge Function	<code>/functions/v1/export-report</code>	Admin report export	JWT + AAL2

Example list request and response

Use `Prefer: count=planned` on larger lists because PostgREST documents that `count=exact` can be slower on large tables, while planned and estimated counts use PostgreSQL statistics. ²²

```
GET /rest/v1/listings?  
select=id,org_id,slug,title,status,published_at&status=eq.published&order=published_at.desc&limit=  
Authorization: Bearer <JWT>  
apikey: <PUBLISHABLE_KEY>  
Prefer: count=planned
```

```
Content-Range: 0-19/1240  
Range-Unit: items
```

```
[  
  {  
    "id": "3b19d99b-ff01-4a65-b0cb-a5ab634d3723",  
    "org_id": "7d358fb7-6bcf-4c1a-a267-69aa193f14a8",  
    "slug": "alpha-hardware",  
    "title": "Alpha Hardware",  
    "status": "published",  
    "published_at": "2026-06-14T12:01:44Z"  
  }  
]
```

Example Edge Function workflow request

```
POST /functions/v1/onboard-org  
Authorization: Bearer <JWT>  
Content-Type: application/json
```

```
{  
  "organization": {  
    "name": "North Coast Suppliers",  
    "slug": "north-coast-suppliers",  
    "billing_email": "billing@northcoast.example"  
  },  
  "listing": {  
    "title": "North Coast Suppliers",  
    "summary": "Industrial parts and field service support",  
    "description": "..."  
  }  
}
```

```
}
}
```

```
{
  "organization_id": "7d358fb7-6bcf-4c1a-a267-69aa193f14a8",
  "owner_membership_id": "0c860532-b85e-4dcc-9c12-a6452c3dc62f",
  "listing_id": "3b19d99b-ff01-4a65-b0cb-a5ab634d3723",
  "status": "ok"
}
```

Validation and concurrency control

Validation should be layered:

- request-schema validation in Edge Functions;
- strict DB constraints for final integrity;
- RLS for authorization;
- optimistic concurrency for user-edited rows.

For `listings`, `ad_campaigns`, and `consultations`, the AI should require `expected_version` on updates and use `where id = $id and version = $expected_version` in the transactional write. If no row is updated, return `409 Conflict`. Realtime payloads should carry the new version so clients can discard stale local state. PostgreSQL constraints remain the durable safety net; API validation is the UX layer.

23

RLS policy examples

Supabase's RLS guidance emphasizes explicit policy scopes, indexes on policy columns, and careful use of `auth.uid()` and `auth.jwt()`. It also notes that `auth.uid()` returns `null` when unauthenticated, which is why policies should clearly target `authenticated` where appropriate. PostgreSQL's row security model is conceptually an implicit `WHERE` clause attached to each query. ²⁴

```
create or replace function private.is_org_member(p_org_id uuid)
returns boolean
language sql
security definer
set search_path = public, auth
as $$
select exists (
  select 1
  from public.organization_memberships m
  where m.org_id = p_org_id
    and m.user_id = auth.uid()
    and m.is_active = true
);
```

```

$$;

create or replace function private.has_permission(p_org_id uuid, p_permission
text)
returns boolean
language sql
security definer
set search_path = public, auth
as $$
    select exists (
        select 1
        from public.organization_memberships m
        join public.role_permissions rp
        on rp.role_code = m.role_code
        where m.org_id = p_org_id
        and m.user_id = auth.uid()
        and m.is_active = true
        and rp.permission_code = p_permission
    );
$$;

alter table public.organizations enable row level security;
alter table public.organization_memberships enable row level security;
alter table public.listings enable row level security;
alter table public.media_assets enable row level security;
alter table public.consultations enable row level security;
alter table public.notifications enable row level security;

create policy "members can read own organizations"
on public.organizations
for select
to authenticated
using (private.is_org_member(id));

create policy "members can read own memberships"
on public.organization_memberships
for select
to authenticated
using (
    user_id = auth.uid()
    or private.has_permission(org_id, 'org.members.read')
);

create policy "admins manage memberships with aal2"
on public.organization_memberships
for all
to authenticated
using (private.has_permission(org_id, 'org.members.manage'))

```

```

with check (
  private.has_permission(org_id, 'org.members.manage')
  and (select auth.jwt()->'aal') = 'aal2'
);

create policy "public can read published listings"
on public.listings
for select
to anon, authenticated
using (status = 'published');

create policy "org members can read unpublished listings"
on public.listings
for select
to authenticated
using (
  private.is_org_member(org_id)
);

create policy "authorized members can write listings"
on public.listings
for all
to authenticated
using (private.has_permission(org_id, 'listings.write'))
with check (private.has_permission(org_id, 'listings.write'));

create policy "org members can access media"
on public.media_assets
for select
to authenticated
using (private.is_org_member(org_id));

create policy "authorized members can upload media"
on public.media_assets
for insert
to authenticated
with check (private.has_permission(org_id, 'media.write'));

create policy "consultation participants can read"
on public.consultations
for select
to authenticated
using (
  requester_user_id = auth.uid()
  or assigned_to_user_id = auth.uid()
  or private.has_permission(org_id, 'consultations.read_all')
);

```

```

create policy "authorized members can update consultations"
on public.consultations
for update
to authenticated
using (
    assigned_to_user_id = auth.uid()
    or private.has_permission(org_id, 'consultations.write')
)
with check (
    assigned_to_user_id = auth.uid()
    or private.has_permission(org_id, 'consultations.write')
);

create policy "users read own notifications"
on public.notifications
for select
to authenticated
using (user_id = auth.uid());

create policy "users update own notifications"
on public.notifications
for update
to authenticated
using (user_id = auth.uid())
with check (user_id = auth.uid());

```

Realtime design and conflict resolution

Supabase supports two database-change subscription paths and recommends Broadcast for scalability and security. Broadcast is authorized with RLS on `realtime.messages`; Postgres Changes require a publication and are easier to start with but not as scalable. Realtime Authorization evaluates policies on channel join, uses JWT plus request headers plus topic information, and Supabase notes that more complex RLS on `realtime.messages` can increase connection latency and reduce join rates. ²⁵

For this system, the best design is:

- Use **Broadcast** for listing status changes, consultation room events, notification fan-out, campaign approval events, and admin presence.
- Use **Presence** for “agent online”, “currently editing listing”, and “consultation room active participants”.
- Limit **Postgres Changes** to low-volume internal admin feeds or debugging tools.

Use topic names with semantic scoping, for example:

- `org:<org_id>:consultations`
- `org:<org_id>:consultation:<consultation_id>`
- `org:<org_id>:listings`

- `org:<org_id>:ads`

For conflict resolution, maintain `version` columns on mutable rows and treat Realtime as advisory transport, not the source of truth. On writes, compare `expected_version`; on successful commit, increment version; on Realtime receipt, discard events older than local state. Presence remains ephemeral and never overwrites authoritative database state.

A representative Realtime authorization policy for consultation rooms would look like this:

```
create policy "consultation participants can receive room broadcasts"
on realtime.messages
for select
to authenticated
using (
  realtime.messages.extension in ('broadcast', 'presence')
  and exists (
    select 1
    from public.consultations c
    where ('org:' || c.org_id::text || ':consultation:' || c.id::text) =
realtime.topic()
    and (
      c.requester_user_id = auth.uid()
      or c.assigned_to_user_id = auth.uid()
      or private.has_permission(c.org_id, 'consultations.read_all')
    )
  )
);

create policy "consultation participants can send room broadcasts"
on realtime.messages
for insert
to authenticated
with check (
  realtime.messages.extension in ('broadcast', 'presence')
  and exists (
    select 1
    from public.consultations c
    where ('org:' || c.org_id::text || ':consultation:' || c.id::text) =
realtime.topic()
    and (
      c.requester_user_id = auth.uid()
      or c.assigned_to_user_id = auth.uid()
      or private.has_permission(c.org_id, 'consultations.write')
    )
  )
);
```

Edge functions, integrations, security, and operations

Supabase Edge Functions are the right execution layer for trusted orchestration. They can perform database work, call third-party APIs, use background tasks through `EdgeRuntime.waitUntil`, and be scheduled using `pg_cron` + `pg_net`. They are also constrained: Supabase documents a 256 MB memory cap, paid-plan wall-clock duration up to 400 seconds, 2 seconds of CPU time per request, and a 150-second request idle timeout. Those limits mean Edge Functions are excellent for short-lived orchestration and webhooks, but not for long batch jobs that should instead use scheduled chunking or an external worker.

26

Recommended Edge Function catalog

Function	Responsibility	Trigger
<code>onboard-org</code>	Create organization, seed roles, default plan, and initial listing	User-initiated
<code>create-checkout-session</code>	Stripe checkout or billing portal session	User-initiated
<code>stripe-webhook</code>	Verify signature, apply billing state changes, write audit events	Stripe webhook
<code>send-notification</code>	Fan-out email/push/in-app notification events	Outbox drain
<code>expire-campaigns</code>	Mark campaigns completed or expired	Scheduled
<code>retry-outbox</code>	Retry failed integration deliveries	Scheduled
<code>export-report</code>	Generate tenant report and store artifact in private bucket	Admin-initiated
<code>rebuild-search</code>	Recompute derived search or reporting state in batches	Scheduled/ admin

Supabase also offers **Database Webhooks**, which are a convenience wrapper around triggers using `pg_net`. That makes them useful for non-critical fire-and-forget side effects. For critical integrations, though, the AI should still prefer an **outbox table** inserted in the same business transaction, then drained by a scheduled function. That gives better retry semantics, observability, and idempotency. 27

Integration choices

Stripe is the most natural payment choice because Supabase publishes a Stripe webhook example for Edge Functions, including signature handling on Deno. Resend is a strong default for app email from Edge Functions, but for Supabase Auth emails themselves the production requirement is different: Supabase explicitly says the default built-in SMTP service is not for production and only delivers to pre-authorized team addresses, so a custom SMTP provider must be configured before launch if email Auth is enabled. 28

For files, use Supabase Storage with private buckets by default. Private buckets are private by default, and the documented ways to download assets are either JWT-authorized download methods or signed URLs with limited lifetime. That is a strong fit for consultation exports, invoices, creatives awaiting approval, and any operational artifact with limited access. ²⁹

For secrets, use Supabase project secrets for Functions and Supabase Vault for database-side scheduled calls or credentials that must be read from SQL. Supabase's function scheduling guide explicitly recommends using Vault to store the auth token or project URL securely for scheduled function invocation.

30

Monitoring and alerting plan

Supabase provides several observability primitives that are sufficient for a real production alerting stack. The platform supports Logs Explorer, log drains to several external systems, a metrics endpoint compatible with Prometheus/OpenMetrics, external dashboards via Grafana Cloud, function invocation logs, auth logs, storage logs, realtime logs, and Postgres statistics via `pg_stat_statements`. Supabase also provides database advisors like the Security Advisor and Index/Performance advisors. ³¹

The recommended monitoring plan is:

Signal	Source	Threshold	Action
REST/RPC latency p95	PostgREST + app traces	> 500 ms for 5 minutes	Investigate hot queries
Edge Function error rate	Function invocations/ logs + Sentry	> 1% for 10 minutes	Page backend owner
Realtime join failure rate	Realtime logs	> 1% for 10 minutes	Inspect <code>realtime.messages</code> policies
Database CPU or I/O saturation	Metrics API / Grafana	> 80% sustained	Consider compute upsize or query tuning
Pooler saturation	Metrics + Supervisor guidance	> 70% sustained	Review pooling and long transactions
Long-running queries	Postgres logs + dashboards	> 5 s	Add index or rewrite query
Failed outbox deliveries	Outbox table + function logs	> 100 pending for > 10m	Retry and inspect provider
Auth anomaly	Auth logs / audit logs	Failure spike or rate-limit spike	Inspect abuse path
Storage egress anomaly	Usage graphs + billing metrics	Daily usage > 20% above baseline	Inspect signed-URL misuse

Concretely, wire it this way:

- Use Supabase Logs Explorer for fast local triage.
- Configure **log drains** to Sentry/Datadog/Loki/S3 depending your standards.
- Use the **metrics API** and Grafana dashboard for DB saturation, connection pools, and platform signals.
- Instrument Edge Functions with **Sentry**.
- Enable and review **pg_stat_statements** weekly.
- Run **Security Advisor / Index Advisor** on a schedule and make fixes part of the sprint. ³²

Security, privacy, and compliance posture

Security best practice on Supabase starts with RLS, private storage by default, minimal use of `service_role`, strong secrets discipline, and transport restrictions where needed. Use the newer signing keys model for JWTs, keep `service_role` out of browsers entirely, protect DB-side secrets with Vault, and lock down database access with network restrictions when you have fixed trusted CIDRs. Supabase documents that network restrictions are enforced before traffic even reaches the database. ³³

For abuse protection, configure Auth rate limits and CAPTCHA where public sign-in, sign-up, magic link, or OTP endpoints are exposed. Supabase supports configurable rate limits and CAPTCHA providers including hCaptcha and Cloudflare Turnstile. For auditability, use Supabase Auth audit logs and your own append-only `audit.activity_log` for business actions. ³⁴

For privacy, keep personal data minimization explicit in the schema, avoid shoving PII into logs or user-agent strings, and define retention windows for consultations, exports, notification payloads, and audit metadata. Supabase's logs guidance explicitly warns developers not to put personal data into `User-Agent` because it can be logged. For file assets, keep sensitive content in private buckets and expose only signed URLs with short expiries. ³⁵

For regulated deployments, region placement and platform add-ons matter. Supabase documents region selection by user proximity; for HIPAA use cases, it documents the need for a signed BAA, the HIPAA add-on, High Compliance project configuration, PITR, SSL enforcement, and network restrictions. ³⁶

Delivery, DX, testing, cost, and timeline

Supabase CLI already gives the developer experience needed for an autonomous implementation loop. The AI should use `supabase init` to bootstrap the project, `supabase start` for local parity, `supabase db reset` to replay migrations and seed the database from `supabase/seed.sql`, `supabase db lint` to catch lint issues, `supabase test db` for pgTAP database tests, and `supabase functions serve` plus Deno tests for local function development. Preview branches should be used per pull request, optionally cloning production data where appropriate for realistic test runs. ³⁷

CI/CD pipeline steps

A production-grade pipeline for this backend should do the following on every PR:

1. Check out the repo.
2. Start the local Supabase stack.
3. Reset the DB from migrations and seed data.
4. Lint SQL and policy code.
5. Run pgTAP tests.
6. Run Deno tests for Edge Functions.
7. Run integration tests against the local stack.
8. Optionally create a preview branch in Supabase for reviewer QA.

On merge to `main`, it should:

1. Require manual approval for production.
2. Apply migrations to the target project.
3. Push secrets.
4. Deploy versioned Edge Functions.
5. Run smoke tests.
6. Flip feature flags only after smoke tests succeed.

This is directly aligned with Supabase's CLI capabilities for local environments, testing, secrets management, and preview branches. ³⁸

Sample GitHub Actions workflow

```
name: supabase-backend

on:
  pull_request:
  push:
    branches: [main]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: supabase/setup-cli@v1
        with:
          version: latest

      - uses: denoland/setup-deno@v2
        with:
          deno-version: v2.x
```

- name: Start local Supabase stack
run: supabase start
- name: Reset local database from migrations and seed
run: supabase db reset --local
- name: Lint database objects
run: supabase db lint --local --fail-on warning
- name: Run pgTAP database tests
run: supabase test db
- name: Run Edge Function tests
run: deno test -A supabase/functions/tests
- name: Run integration tests
run: npm ci && npm run test:integration

preview:

```

if: github.event_name == 'pull_request'
needs: validate
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - uses: supabase/setup-cli@v1
    with:
      version: latest

  - name: Create or update preview branch
    env:
      SUPABASE_ACCESS_TOKEN: ${ secrets.SUPABASE_ACCESS_TOKEN }
    run: |
      supabase branches create pr-${ github.event.pull_request.number } \
        --project-ref "${ secrets.SUPABASE_PROJECT_REF }" \
        --persistent

```

deploy:

```

if: github.ref == 'refs/heads/main'
needs: validate
runs-on: ubuntu-latest
environment: production
concurrency:
  group: prod-supabase
  cancel-in-progress: false
steps:
  - uses: actions/checkout@v4

```

```

- uses: supabase/setup-cli@v1
  with:
    version: latest

- name: Apply migrations
  run: supabase db push --db-url "${{ secrets.SUPABASE_DB_URL }}"

- name: Push function secrets
  env:
    SUPABASE_ACCESS_TOKEN: "${{ secrets.SUPABASE_ACCESS_TOKEN }}"
  run: |
    supabase secrets set \
      STRIPE_API_KEY="${{ secrets.STRIPE_API_KEY }}" \
      STRIPE_WEBHOOK_SECRET="${{ secrets.STRIPE_WEBHOOK_SECRET }}" \
      RESEND_API_KEY="${{ secrets.RESEND_API_KEY }}" \
      SENTRY_DSN="${{ secrets.SENTRY_DSN }}" \
      --project-ref "${{ secrets.SUPABASE_PROJECT_REF }}"

- name: Deploy Edge Functions
  env:
    SUPABASE_ACCESS_TOKEN: "${{ secrets.SUPABASE_ACCESS_TOKEN }}"
  run: |
    for fn in onboard-org create-checkout-session stripe-webhook send-
notification export-report retry-outbox; do
      supabase functions deploy "$fn" --project-ref "$
{{ secrets.SUPABASE_PROJECT_REF }}"
    done

- name: Run post-deploy smoke tests
  run: npm ci && npm run test:smoke

```

Migration and rollback procedure

Supabase supports migration-driven local development and preview branches, and its backup model matters for rollback planning. Paid plans get daily backups; PITR is an add-on; if PITR is enabled it replaces daily backups; and restoring a backup makes the project inaccessible for the duration of the restore. Supabase also documents that database backups do **not** include Storage object contents, only database data and metadata. ³⁹

That implies the rollback strategy should be:

Scenario	Preferred response
Bug found in PR or preview branch	Fix migration and rerun branch preview
Non-destructive prod bug	Deploy a forward-fix migration and/or previous function version

Scenario	Preferred response
Destructive prod data bug	Restore from PITR or latest daily backup, preferably to a new project first
Storage object loss	Restore from separate object backup/versioning; DB restore is not enough
Function regression	Redeploy previous function version from tagged artifact

The safe migration rules for the AI are:

- never edit an already-applied production migration;
- prefer additive changes first;
- use dual-write/backfill/cutover/drop for destructive refactors;
- version function contracts;
- keep seeds deterministic;
- run smoke tests immediately after deploy;
- keep object storage versioning or external replication for critical assets because DB restore will not recover them. ⁴⁰

Testing matrix and sample cases

Supabase supports pgTAP-based database tests through `supabase test db`, and documents Deno-based testing for Edge Functions. The best production test strategy is therefore layered rather than monolithic. ⁴¹

Layer	Tooling	Goal	Example cases
Unit	Deno test, app unit tests	Logic correctness	Schema validation, Stripe signature handling, DTO transforms
Database integration	pgTAP + local Supabase	Integrity and RLS	Unpublished listing invisible to public; cross-tenant read denied
Function integration	Local stack + Deno	End-to-end workflows within Supabase	<code>onboard-org</code> creates org, membership, default listing in one transaction
End-to-end	Browser/mobile test runner	User journeys	Sign up, create listing, upload media, submit consultation, approve ad
Load	k6 or similar	Saturation and latency	Listing search, consultation room fan-out, webhook bursts
Security regression	Policy tests + abuse scenarios	No privilege escalation	Member cannot elevate own role; <code>service_role</code> never exposed client-side

Representative sample test cases the AI should implement immediately:

Test case	Expected result
Public requests <code>/rest/v1/listings?status=eq.draft</code>	0 rows returned
Org A member queries Org B consultation	Access denied by RLS
Member without <code>org.members.manage</code> updates membership role	Access denied
Admin without <code>aal2</code> attempts role change	Access denied
Duplicate Stripe webhook event arrives twice	Only one billing state change persists
Listing update submitted with stale <code>version</code>	<code>409 Conflict</code>
Realtime topic join for unauthorized consultation room	Join denied
Signed URL for private asset after expiry	Download rejected

Cost estimate model and optimization recommendations

Supabase billing combines a plan subscription fee with variable or per-project usage. The billing docs expose the metered components clearly enough to build a production model: per-project compute, database disk overage, egress, storage, Edge Function invocations, Realtime messages, Realtime peak connections, MAU/SSO users, and PITR. For plan-fee accuracy, use the live pricing page at execution time; below, that fixed fee is represented as `F_plan`. ⁴²

A practical monthly model is:

```
Total monthly cost
= F_plan
+ DatabaseCompute
+ DatabaseDiskOverage
+ EgressOverage
+ StorageOverage
+ RealtimeMessageOverage
+ RealtimePeakConnectionOverage
+ EdgeFunctionOverage
+ PITR
+ Optional SSO/phone MFA/log-drain add-ons
```

Current documented usage rates useful for the model include:

- database disk overage at \$0.125/GB-month beyond included disk;
- egress overage at \$0.09/GB beyond included quota;
- storage overage at \$0.0213/GB-month;
- Realtime messages at \$2.50 per 1 million over quota;
- Realtime peak connections at \$10 per 1,000 over quota;

- Edge Function invocations at \$2 per 1 million over quota;
- PITR at \$100/month for 7 days, \$200/month for 14 days, and \$300/month for 28 days. ⁴³

A **worked moderate-scale example** for 100k users and 10k peak concurrent connections might look like this, **excluding** the base organization plan fee:

Component	Example assumption	Estimated monthly cost
Database compute	Large	\$110.00
Database disk overage	100 GB total, 8 GB included	\$11.50
Storage overage	300 GB total, 100 GB included	\$4.26
Egress overage	1 TB total, 250 GB included	\$69.66
Realtime messages	20M total, 5M included	\$37.50
Realtime peak connections	10,000 total, 500 included	\$95.00
Edge Functions	8M total, 2M included	\$12.00
PITR	7-day retention	\$100.00
Subtotal before plan fee		\$439.92

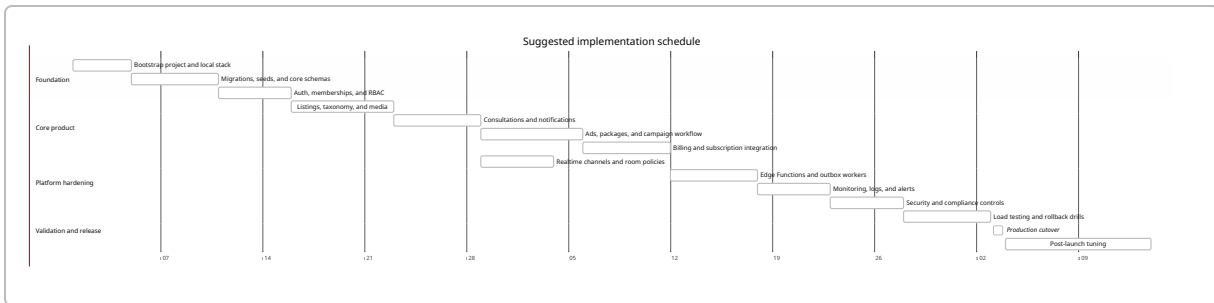
This example is directionally sound, but two caveats matter. First, 10k peak concurrent connections require Pro without spend cap or a higher plan according to current Realtime limits. Second, actual DB compute size should be chosen from observed CPU, I/O, and lock behavior rather than from user count alone. ⁴⁴

The AI should optimize cost in this order:

1. Use Broadcast instead of Postgres Changes for high-volume Realtime.
2. Keep large media in Storage, not Postgres.
3. Tune queries and indexes before upsizing compute.
4. Keep private buckets truly private and signed URLs short-lived to reduce accidental hot-linking.
5. Batch non-urgent jobs in scheduled functions instead of synchronous request-time work.
6. Use planned/estimated counts on large lists instead of exact counts where UX allows.
7. Review advisors and `pg_stat_statements` weekly. ⁴⁵

Implementation timeline

A realistic autonomous rollout for this backend is about twelve weeks if the AI has a clear spec, repository permissions, and access to a Supabase project.



The practical milestone gates should be:

Milestone	Entry criteria	Exit criteria
Foundation complete	Local stack booting	Migrations replay from zero; seeds deterministic
Auth complete	Profiles and orgs created	RLS tests pass; role escalation blocked
Core product complete	Listings and consultations implemented	Media uploads, notifications, and tenant isolation verified
Billing complete	Stripe test account wired	Idempotent webhook handling verified
Ops complete	Metrics/logs reachable	Alert rules sending to real destination
Release ready	All tests green	PITR/backups configured; smoke tests and rollback drill completed

The AI should not promote to production until the following are true: every migration replays from scratch, preview branches are working, RLS denial tests are passing, webhook idempotency has been proven, Auth email is no longer using the default SMTP path, and either daily backups or PITR is enabled. ⁴⁶

¹ ⁹ ¹⁷ ¹⁹ ²⁰ Auth | Supabase Docs

https://supabase.com/docs/guides/auth?utm_source=chatgpt.com

² ¹⁰ ²¹ Tables and Views — PostgreSQL 14 documentation

https://postgresql.org/en/stable/references/api/tables_views.html?utm_source=chatgpt.com

³ ⁶ ²⁴ Row Level Security | Supabase Docs

https://supabase.com/docs/guides/database/postgres/row-level-security?utm_source=chatgpt.com

⁴ ⁷ ³⁷ ³⁸ CLI Reference | Supabase Docs

https://supabase.com/docs/reference/cli/introduction?utm_source=chatgpt.com

⁵ ³⁶ Available regions | Supabase Docs

https://supabase.com/docs/guides/platform/regions?utm_source=chatgpt.com

⁸ ⁴² ⁴³ Control your costs | Supabase Docs

https://supabase.com/docs/guides/platform/cost-control?utm_source=chatgpt.com

- 11 25 45 **Subscribing to Database Changes | Supabase Docs**
https://supabase.com/docs/guides/realtime/subscribing-to-database-changes?utm_source=chatgpt.com
- 12 **Storage Quickstart | Supabase Docs**
https://supabase.com/docs/guides/storage/quickstart?utm_source=chatgpt.com
- 13 15 23 **PostgreSQL: Documentation: 18: 5.5. Constraints**
https://www.postgresql.org/docs/current/ddl-constraints.html?utm_source=chatgpt.com
- 14 **User Management | Supabase Docs**
https://supabase.com/docs/guides/auth/managing-user-data?utm_source=chatgpt.com
- 16 **Auth architecture | Supabase Docs**
https://supabase.com/docs/guides/auth/architecture?utm_source=chatgpt.com
- 18 **JWT Claims Reference | Supabase Docs**
https://supabase.com/docs/guides/auth/jwt-fields?utm_source=chatgpt.com
- 22 **Pagination and Count — PostgREST 14 documentation**
https://postgrest.org/en/stable/references/api/pagination_count.html?utm_source=chatgpt.com
- 26 **Background Tasks | Supabase Docs**
https://supabase.com/docs/guides/functions/background-tasks?utm_source=chatgpt.com
- 27 **Database Webhooks | Supabase Docs**
https://supabase.com/docs/guides/database/webhooks?utm_source=chatgpt.com
- 28 **Handling Stripe Webhooks | Supabase Docs**
https://supabase.com/docs/guides/functions/examples/stripe-webhooks?utm_source=chatgpt.com
- 29 **Storage Buckets | Supabase Docs**
https://supabase.com/docs/guides/storage/buckets/fundamentals?utm_source=chatgpt.com
- 30 **Scheduling Edge Functions | Supabase Docs**
https://supabase.com/docs/guides/functions/schedule-functions?utm_source=chatgpt.com
- 31 35 **Logging | Supabase Docs**
https://supabase.com/docs/guides/telemetry/logs?utm_source=chatgpt.com
- 32 **Log Drains | Supabase Docs**
https://supabase.com/docs/guides/telemetry/log-drains?utm_source=chatgpt.com
- 33 **JWT Signing Keys | Supabase Docs**
https://supabase.com/docs/guides/auth/signing-keys?utm_source=chatgpt.com
- 34 **Rate limits | Supabase Docs**
https://supabase.com/docs/guides/auth/rate-limits?utm_source=chatgpt.com
- 39 40 **Database Backups | Supabase Docs**
https://supabase.com/docs/guides/platform/backups?utm_source=chatgpt.com
- 41 46 **Testing Your Database | Supabase Docs**
https://supabase.com/docs/guides/database/testing?utm_source=chatgpt.com
- 44 **Realtime Limits | Supabase Docs**
https://supabase.com/docs/guides/realtime/limits?utm_source=chatgpt.com